

Big Data

Big Data refers to extremely large and complex sets of data that are **too vast, fast, or varied** for traditional data processing tools (like relational databases) to handle efficiently.

Definition

Big Data is data that is:

- **Too large** in volume (terabytes, petabytes, or more),
- **Generated at high speed** (real-time or near real-time), and
- **Comes in many forms** (structured, semi-structured, and unstructured).

It requires **specialized tools, technologies, and frameworks** like:

- **Hadoop**
- **Spark**
- **Kafka**
- **Azure Data Lake**
- **AWS EMR**
- **Google BigQuery**

The 5 V’s of Big Data

V	Description	Example
Volume	Huge amount of data generated every second	Social media posts, IoT sensors
Velocity	Speed at which data is created, processed, and analyzed	Stock market feeds, real-time ads
Variety	Different formats of data	Text, images, videos, logs
Veracity	Quality or accuracy of data	Removing noise, duplicate data
Value	Insights or benefits gained from analyzing data	Predicting customer behavior

Types of Big Data

1. **Structured Data** → Organized (rows/columns)
Example: SQL databases, spreadsheets
2. **Unstructured Data** → No fixed format
Example: Videos, social media posts, emails
3. **Semi-structured Data** → Partially organized
Example: JSON, XML, logs

Big Data Technologies

Category	Examples
Storage	HDFS, Azure Data Lake, AWS S3
Processing	Hadoop MapReduce, Apache Spark
Streaming	Apache Kafka, Azure Event Hubs
Querying	Hive, Presto, Databricks SQL
Visualization	Power BI, Tableau

Real-World Examples

- **Netflix** uses big data to recommend shows based on user behavior.
- **Banks** use it to detect fraud in real-time.
- **Healthcare** uses it for predictive diagnosis.
- **Retail** uses it for inventory and customer trend analysis.

In Azure Context

In **Microsoft Azure**, Big Data is handled using:

- **Azure Data Lake Storage (ADLS)**
- **Azure Synapse Analytics**
- **Azure Databricks**
- **HDInsight (Hadoop on Azure)**
- **Azure Stream Analytics**

History

How to scale massive amount of data

Google released just a paper

Google File system	2003
Map Reduce	2004

But few persons from yahoo implemented these papers and they created one framework called **HADOOP**

Later its given to **apache software**. Open source platform

First version of Hadoop was released in 2006.

HADOOP file system was released HDFS (hadoop distributed file system)

Process solution was **MAP REDUCE** retained this name from papers

Apache Spark

1. Origin (2009–2010) – Birth at UC Berkeley

➤ To overcome the **limitations of Hadoop MapReduce**, which was powerful but **slow** for iterative and interactive workloads.

- Spark was designed to process data **in-memory** (RAM), drastically improving performance over Hadoop's disk-based approach.
-

Key Idea:

“Keep data in memory between computations instead of writing to disk after every step.”

2. Open Source Release (2010)

- Spark was **open-sourced in 2010** under a **BSD license**.

3. Apache Foundation Era (2013)

- In **2013**, Spark was **donated to the Apache Software Foundation (ASF)**.
- It became an official **Apache Incubator project** and soon graduated to a **Top-Level Project** in **2014** due to its growing popularity.

Reason for Popularity:

- Unified engine for **batch + streaming + ML + graph** workloads.
- Easier API compared to Hadoop (supporting **Scala, Java, Python, R**).
- Much **faster performance** (up to 100x faster than Hadoop for some tasks).

Big data processing On Cloud

Feature	Azure	AWS	Google Cloud
Storage	ADLS	S3	BigLake
Compute/Processing	Databricks, Synapse	EMR, Glue	Dataproc, Dataflow
Analytics	Synapse	Redshift, Athena	BigQuery
Streaming	Event Hub, Stream Analytics	Kinesis	Pub/Sub
Machine Learning	Azure ML	SageMaker	Vertex AI
Visualization	Power BI	QuickSight	Looker Studio
Governance	Purview	Lake Formation	Data Catalog

Distributed Computing

Distributed Computing is one of the **core foundations** of *Big Data, Cloud, and Spark*. Let’s go step by step so it’s crystal clear.

What is Distributed Computing?

Distributed Computing means splitting a large computational task into **smaller subtasks**, running them **simultaneously across multiple computers (nodes)**, and then combining the results.

In short:

“It’s teamwork between computers — many systems working together to solve one big problem.”

Simple Example

Imagine you have to count the words in **1 billion files**:

- On a **single computer**, it might take weeks.
- In **distributed computing**, you divide files among **100 computers**, each counts part of the data, and then results are merged — done in hours or minutes.

Key Concepts

Term	Description
Node	A single computer (server or VM) in the distributed system
Cluster	A group of interconnected nodes working together
Master Node	Coordinates and distributes tasks (manager)
Worker Nodes	Perform actual computations (workers)
Parallel Processing	Executing multiple tasks simultaneously
Fault Tolerance	System continues even if one node fails
Scalability	Easily add more nodes for larger workloads

How Distributed Computing Works

- 1. **Divide the Task** – Break the large job into smaller chunks.
- 2. **Distribute the Task** – Send chunks to multiple worker nodes.
- 3. **Process in Parallel** – Each node processes its data simultaneously.
- 4. **Combine Results** – Gather and merge all outputs into a final result.

Example: Distributed Word Count

- 1. Split 100 GB file into 10 parts → 10 nodes process in parallel
- 2. Each node counts words in its portion
- 3. Master node combines counts → Final total

Basic Terminology used in Big data processing - another ppt

Major component of the Hadoop ecosystem, clearly summarized for quick recall and interview use

Hadoop Ecosystem – One-Line Answers

Component	One-Line Description
HDFS (Hadoop Distributed File System)	Stores large data files across multiple nodes with replication for fault tolerance.
YARN (Yet Another Resource Negotiator)	Manages and schedules cluster resources and job execution in Hadoop.
MapReduce	Framework for processing large data sets in parallel across a Hadoop cluster.
Hive	Data warehouse tool that converts SQL-like queries (HiveQL) into MapReduce jobs.

Pig	High-level scripting language (Pig Latin) for data transformation and analysis.
HBase	NoSQL distributed database for storing structured data on top of HDFS.
Sqoop	Tool for transferring data between Hadoop and relational databases.
Flume	Used to collect, aggregate, and move large amounts of log data into HDFS.
Oozie	Workflow scheduler to manage and automate Hadoop jobs.
Zookeeper	Centralized coordination service for managing distributed applications.
Mahout	Machine learning library built for scalable data mining on Hadoop.
Spark	In-memory data processing engine faster than MapReduce for batch and stream processing.
Ambari	Web-based tool for provisioning, managing, and monitoring Hadoop clusters.
Kafka	Distributed publish-subscribe messaging system for real-time data streaming.
Hue	Web-based user interface for interacting with Hadoop components (like Hive, Pig, etc.).
Avro	Data serialization system for compact and fast data exchange in Hadoop.
Parquet	Columnar storage format optimized for analytics and query performance.
ZooKeeper	Manages configuration and synchronization for distributed systems.
NiFi	Data flow automation tool for moving and transforming data between systems.
Atlas	Data governance and metadata management tool in the Hadoop ecosystem.

In **Big Data** and **Cloud Data Engineering**, understanding the **types of data processing** is key to designing efficient data solutions.

Types of Data Processing

1. Batch Processing

- **Definition:** Processing large volumes of data collected over a period of time — all at once.
- **Key Feature:** Data is processed in **batches or chunks** at scheduled intervals (e.g., hourly, daily).
- **Use Case:** Payroll systems, monthly reports, billing, data warehouse loads.
- **Tools:**
 - Hadoop MapReduce
 - Apache Spark (Batch mode)
 - Azure Data Factory
 - AWS Glue
 - Azure Synapse Pipelines
 -

Example:

Collecting daily sales transactions and generating a report at midnight.

2. Stream (or Real-Time) Processing

- **Definition:** Processing data **as soon as it's generated or received**, in real-time or near-real-time.
- **Key Feature:** Continuous, event-by-event processing.
- **Use Case:** Fraud detection, IoT sensor monitoring, real-time analytics dashboards.
- **Tools:**
 - Apache Kafka
 - Azure Stream Analytics
 - Apache Flink
 - Apache Spark Streaming
 - AWS Kinesis

Example:

Analyzing credit card transactions instantly to detect fraudulent activity.

3. Micro-Batch Processing

- **Definition:** A **hybrid approach** that processes small batches of data at very short intervals (e.g., every few seconds).
- **Key Feature:** Combines benefits of batch (reliability) and stream (speed).
- **Use Case:** Real-time analytics with small time delays.
- **Tools:**
 - Apache Spark Structured Streaming
 - Azure Databricks
 - Google Dataflow

Example:

Processing streaming website click data every 5 seconds for live dashboards.

4. Interactive (Ad-hoc) Processing

- **Definition:** Processing data **on demand** in response to user queries.
- **Key Feature:** Used for exploration, BI, and quick decision-making.
- **Use Case:** Data analysts querying data interactively for insights.
- **Tools:**
 - Hive (LLAP)
 - Presto / Trino
 - Azure Synapse Serverless SQL
 - Google BigQuery
 -

Example:

A user running an instant SQL query to see today’s top-selling product.

5. Near Real-Time Processing

- **Definition:** Data is processed within a few seconds or minutes after arrival.
- **Key Feature:** Slight delay (latency) is acceptable compared to strict real-time systems.
- **Use Case:** Social media analytics, log monitoring, IoT dashboards.
- **Tools:**
 - Azure Event Hub + Stream Analytics
 - Kafka Streams
 - Spark Streaming

Example:

Twitter analytics updating every 2 minutes.

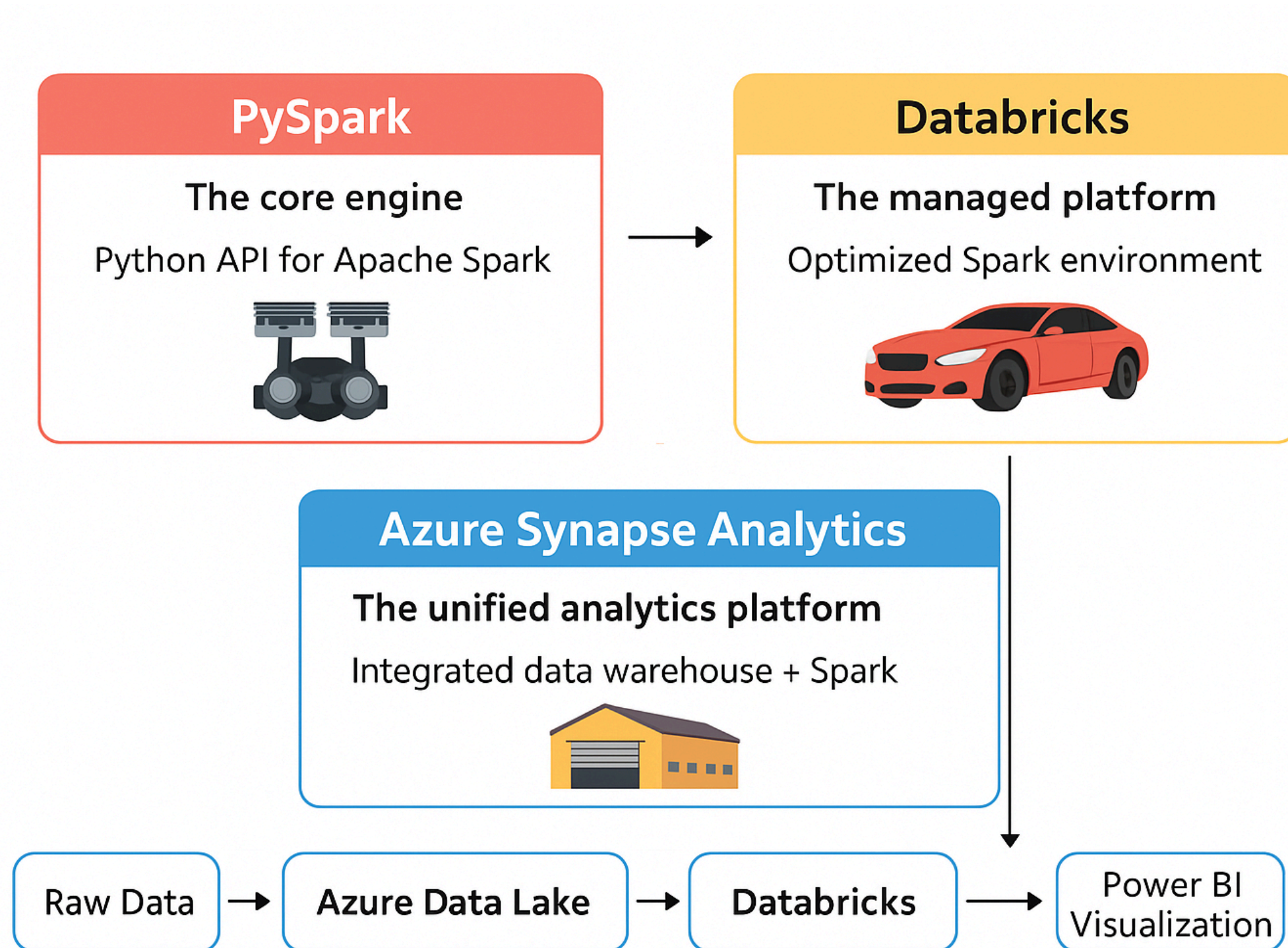
Summary Table

Type	Timing	Example	Tools
Batch Processing	Periodic (hours/days)	Payroll report	Hadoop, ADF, Glue
Stream Processing	Continuous (real-time)	Fraud detection	Kafka, Flink, Stream Analytics
Micro-Batch Processing	Every few seconds	Website clicks	Spark Structured Streaming
Interactive Processing	On demand	BI query	Presto, Synapse, BigQuery
Near Real-Time	Seconds to minutes delay	IoT monitoring	Kafka, Event Hub

What is PySpark, Databricks, and Azure Synapse Analytics,

PySpark vs Databricks vs Azure Synapse Analytics

These three are among the biggest names in the modern **big data ecosystem**. They're often mentioned together but serve **different, complementary roles** in a complete Azure data engineering setup.



1. PySpark — The Core Engine

Analogy: PySpark is the *engine of a car*.

What It Is

PySpark is the **Python API for Apache Spark**, a powerful open-source distributed computing framework.

It's the **programming toolkit** you use to write code that processes large-scale data across multiple computers.

Primary Role

Perform **big data processing and analytics** — both **batch** and **streaming** — across distributed clusters.

Key Characteristics

- **Language:** Python (runs on top of Spark's JVM engine written in Scala)
- **Core Concept:** Uses **RDDs (Resilient Distributed Datasets)** and **DataFrames** for distributed data operations.
- **Lazy Evaluation:** Transformations are not executed until an action (like `.count()` or `.show()`) is triggered.
- **Use Cases:** ETL pipelines, data cleaning, aggregations, stream processing, and machine learning.

In Short:

PySpark is the **programming engine** — you use it to *tell a cluster* how to process and transform your big data.

2. Databricks — The Managed Platform for Spark

Analogy: Databricks is a **luxury car** with the PySpark engine pre-installed, maintained by expert mechanics, with a powerful dashboard.

What It Is

Databricks is a **cloud-based, unified data platform** built by the creators of Apache Spark. It simplifies how you build, manage, and scale big data and AI solutions.

Primary Role

A **collaborative workspace** for data engineers, scientists, and analysts to run PySpark and other frameworks efficiently — without managing infrastructure.

Key Characteristics

- **Managed Spark:** Databricks handles all **cluster setup, scaling, and optimization**.
- **Collaborative Notebooks:** Integrated workspace for Python, SQL, Scala, and R.
- **Delta Lake:** Built-in **ACID-compliant storage layer** providing versioning, schema enforcement, and upserts (MERGE).
- **Workflows:** Built-in tools to **schedule, monitor, and automate** ETL pipelines.
- **MLflow Integration:** End-to-end machine learning experiment tracking and model management.

In Short:

Databricks is a **premium, managed environment** to run your PySpark code — with tools for reliability, performance, and collaboration.

3. Azure Synapse Analytics — The Unified Analytics Platform

Analogy: Azure Synapse is a *massive logistics and warehouse system* for your data — with specialized departments, one of which runs on the PySpark engine.

What It Is

Azure Synapse Analytics is Microsoft’s **end-to-end analytics service** that unifies:

- Data integration (like ADF),
- Big data analytics (via Spark),
- Enterprise data warehousing (via SQL pools).

Primary Role

An **integrated analytics environment** that allows you to ingest, prepare, manage, and serve data for business intelligence and analytics.

END

Key Characteristics

- **Converged Platform:** Combines multiple engines:
 - **SQL Pool** → MPP data warehouse for T-SQL queries.
 - **Spark Pool** → Fully managed **Apache Spark** (where PySpark runs).
 - **Data Explorer Pool** → For real-time analytics and log data.
 - **Pipelines** → Built-in ETL/ELT tool (based on Azure Data Factory).
- **Unified Interface:** Synapse Studio provides one workspace to manage all.
- **Serverless SQL:** Query data directly from ADLS without data movement.
- **Azure Integration:** Deeply integrated with **Power BI**, **Azure ML**, and **Purview**.

In Short:

Synapse is a **comprehensive analytics warehouse** that brings SQL, Spark, and pipelines together — all within the Azure ecosystem.

Summary: PySpark vs Databricks vs Synapse

Feature	PySpark	Databricks	Azure Synapse Analytics
What It Is	Python API for Apache Spark	Managed Spark-based platform	Unified analytics and warehousing service

Core Role	Write big data processing logic	Run, scale, and automate Spark workloads	Integrate data warehousing, Spark, and ETL
Primary Strength	Flexibility and control	Delta Lake, performance, collaboration	Unified SQL + Spark + Data Integration
Cluster Management	Manual setup	Fully managed by Databricks	Fully managed by Microsoft
Key Differentiator	Core open-source engine	Delta Lake + MLflow + Photon optimization	Deep Azure integration + SQL Pools
Best For	Developers learning Spark	Production ETL & ML pipelines	Enterprise data warehousing and analytics

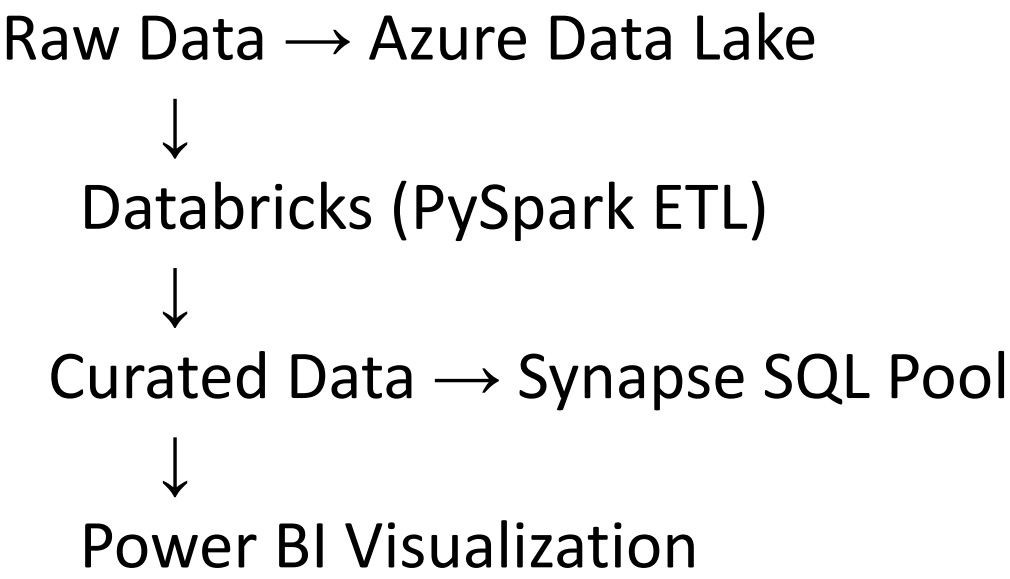
How They Work Together

These tools aren’t competitors — they often **work hand-in-hand** in a modern Azure data architecture.

Typical Flow:

- 1. **PySpark** → Used to write transformation logic.
- 2. **Databricks** → Executes PySpark efficiently in a managed environment.
- 3. **Azure Synapse** → Stores and analyzes the processed data for BI and reporting.

Example Flow:

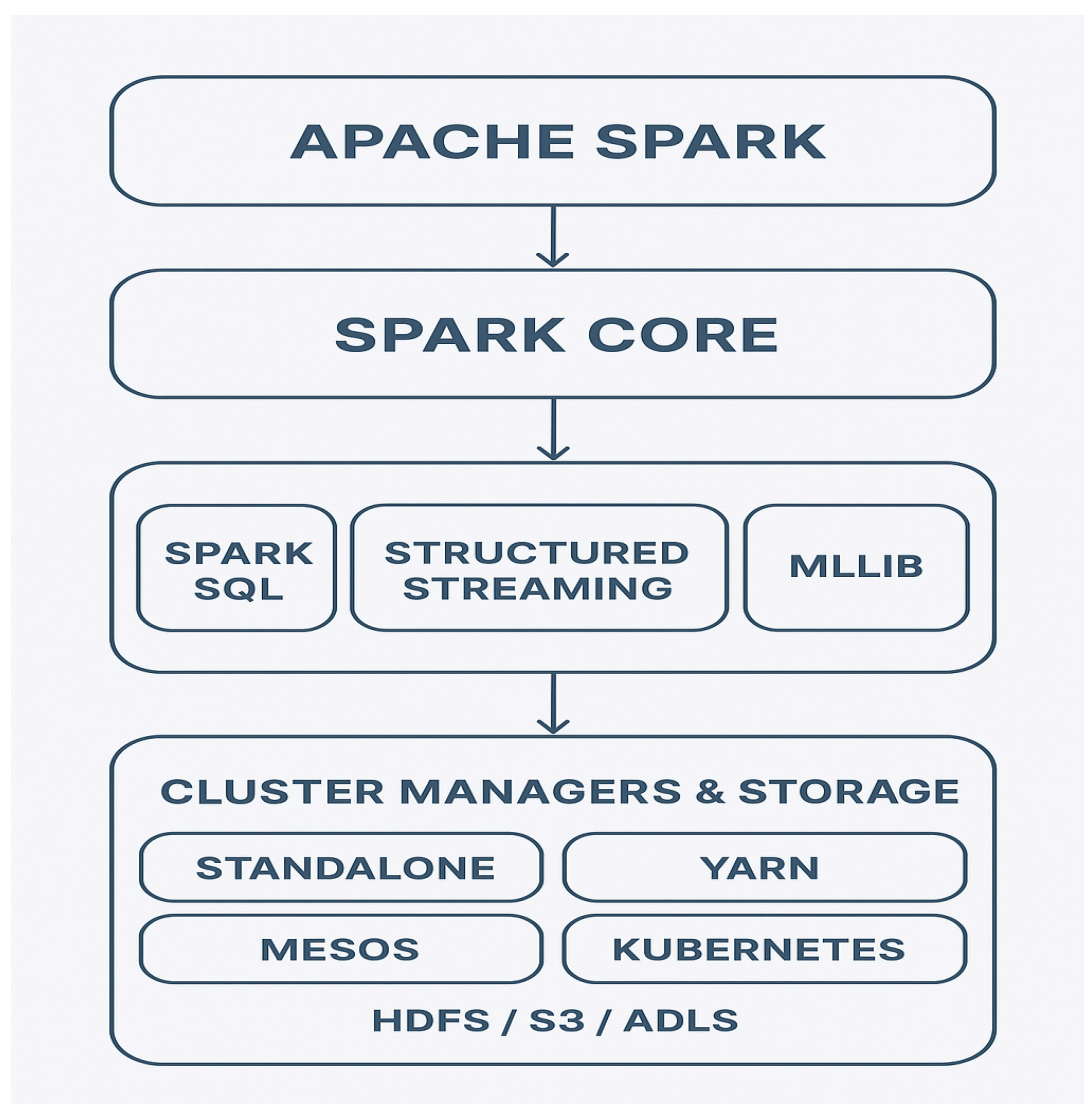


Final Analogy Recap

Concept	Analogy
PySpark	The engine blueprint — powerful but raw.
Databricks	The luxury car with that engine — optimized, managed, and ready to drive.
Azure Synapse	The entire automotive company — offering a full suite of vehicles and systems (warehousing, Spark, pipelines).

Apache Spark Architecture & Components

Apache Spark is a **unified analytics engine** for **large-scale data processing**. Its power lies in a **modular architecture**, where specialized libraries are built on top of a common foundation — **Spark Core**.



1. The Foundation: Spark Core

Spark Core is the **base engine** of the Spark ecosystem. All other Spark components — SQL, Streaming, MLlib, GraphX — are built on top of it.

Key Responsibilities

- **Task Scheduling:** Distributes computation across the cluster.
- **Memory Management:** Handles in-memory and disk-based data storage.
- **Fault Tolerance:** Recovers automatically from failures using **RDDs (Resilient Distributed Datasets)**.
- **Storage Interaction:** Connects with various storage systems like
 - o HDFS
 - o Apache Cassandra
 - o Amazon S3
 - o Azure Blob Storage
 - o Local file systems

Think of Spark Core as the operating system of Spark — it manages all low-level execution and coordination.

2. Specialized Libraries Built on Spark Core

The true strength of Spark comes from its **modular ecosystem**.

Each library specializes in a different type of data workload — all sharing the same execution engine and cluster.

(a) Spark SQL (and DataFrames)

Purpose: Structured and semi-structured data processing.

Abstraction: **DataFrames** and **Datasets** — higher-level APIs built on top of RDDs.

Key Features

- **SQL Query Support:** Run SQL queries natively inside Spark.
- **Catalyst Optimizer:** Automatically optimizes DataFrame and SQL query execution plans.
- **Tungsten Engine:** Advanced memory and CPU optimization for faster performance.
- **Unified Data Access:** Supports multiple data sources — JSON, Parquet, ORC, Avro, Hive, JDBC, etc.

Use Case:

ETL (Extract-Transform-Load) pipelines, batch transformations, and analytics.

(b) Spark Streaming

Purpose: Real-time and near-real-time data processing.

Abstraction:

- **DStream (Discretized Stream):** Original micro-batch API (built on RDDs).
- **Structured Streaming:** Modern API built on **DataFrames/Datasets** for simpler, scalable stream processing.
-

How It Works

- Converts real-time data streams into **micro-batches**.
- Each batch is processed using Spark Core's execution engine.

Use Case:

Live log analysis, IoT sensor data, clickstream analytics, and fraud detection.

(c) MLlib (Machine Learning Library)

Purpose: Scalable machine learning built on distributed data.

Key Features

- **Algorithms:** Classification, regression, clustering, and collaborative filtering.
- **Feature Engineering:** Tools for feature extraction, transformation, and dimensionality reduction.
- **Pipelines:** High-level API for training, evaluating, and tuning ML models (similar to scikit-learn).

Use Case:

End-to-end machine learning workflows at scale — from data prep to model deployment.

GraphX

Purpose: Graph analytics and computation.
Abstraction: Property Graph — vertices and edges with attached attributes.

Key Features

- Built-in algorithms (PageRank, Connected Components, Triangle Counting, etc.).
- Operators for custom graph transformations and joins.

Use Case:

Social network analysis, fraud detection, route optimization, and network topology studies.

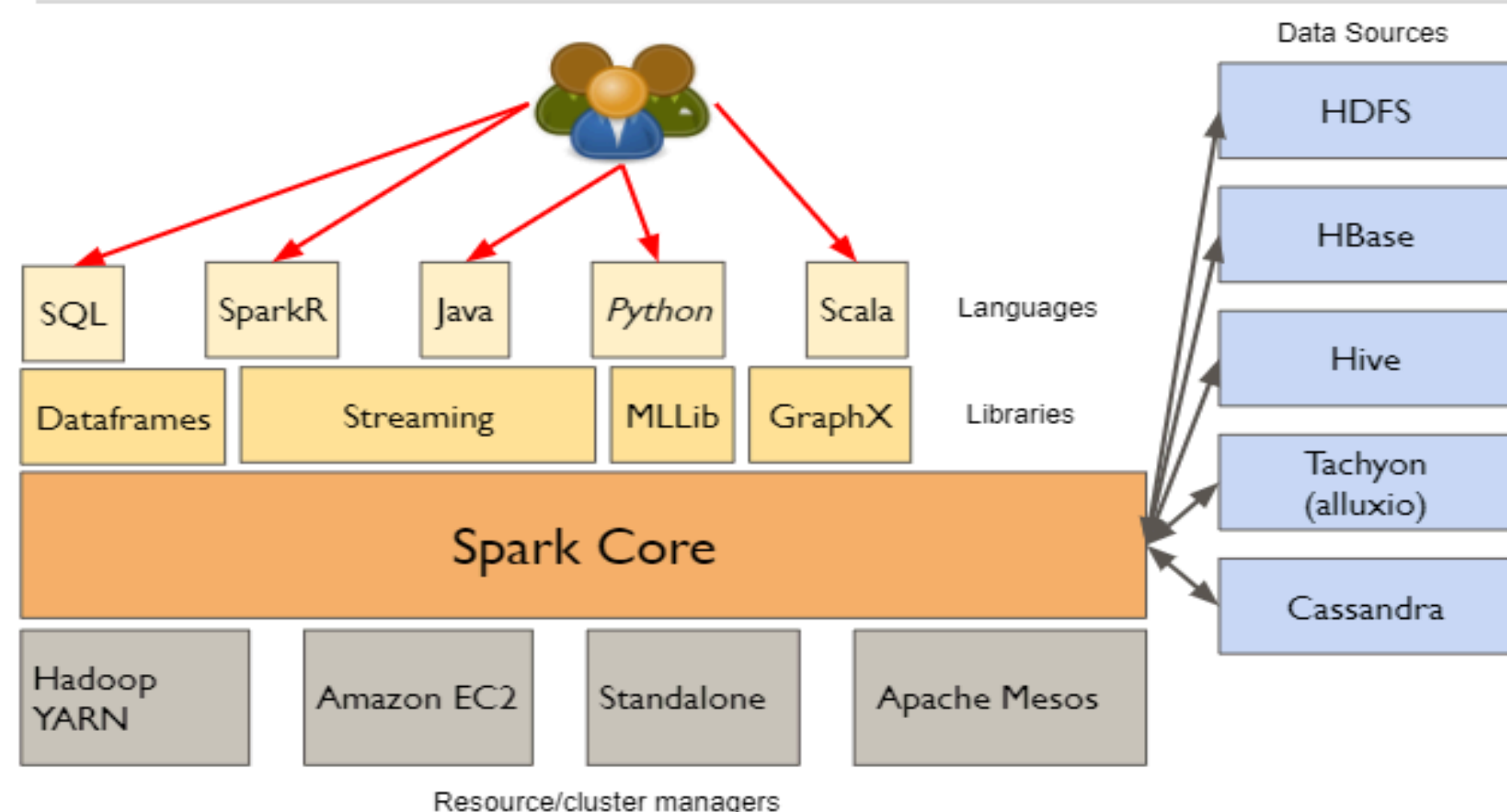
3. Cluster Managers

Spark Core interacts with **cluster managers** to request and manage computing resources. These are not part of Spark itself but essential for distributed operation.

Cluster Manager	Description
Standalone	Spark’s built-in simple cluster manager (default for small clusters).
YARN	Hadoop’s resource manager, common in on-prem Hadoop setups.
Mesos	General-purpose cluster manager (less common today).
Kubernetes	Modern, container-based manager ideal for cloud environments.

4. Visual Overview: Apache Spark Ecosystem

Spark Architecture



5. Summary

Layer	Component	Description
Core Engine	Spark Core	The foundation for distributed execution and fault tolerance.
Structured Processing	Spark SQL	Processes structured/semi-structured data via DataFrames and SQL.
Streaming	Spark Streaming	Real-time and micro-batch data processing.
Machine Learning	MLlib	Distributed machine learning algorithms and pipelines.
Graph Analytics	GraphX	Graph-parallel computation library.
Resource Management	Cluster Managers	Allocate resources across nodes.

Key Takeaway

Apache Spark's modular design lets you build end-to-end analytics — from ETL to ML to streaming — all in one unified engine.

Each component (SQL, Streaming, MLlib, GraphX) shares the same **Spark Core** foundation and runs on the same cluster.

Understanding a Spark Cluster

A **Spark Cluster** is the foundation that allows Apache Spark to process massive datasets in parallel across multiple computers.

Think of it like a team of workers:

- **Single Machine (Non-Cluster):** Like one person trying to build a car alone. It's slow and limited by that person's strength and time.

- **Spark Cluster:** Like an entire factory assembly line. Many workers (computers) specialize in different tasks and work together simultaneously to build the car much faster.

Core Components of a Spark Cluster

A Spark Cluster follows a **master-slave architecture**.

Here are the key components:

1. Cluster Manager (The Master / The Boss)

This is the central coordinator that manages resources and applications across the cluster.

Role:

It's responsible for allocating resources (CPU, memory) to the various Spark applications that want to run on the cluster.

Examples: Spark can run on several cluster managers:

- **Standalone Cluster Manager:** Spark's own built-in, simple cluster manager. Good for getting started.
- **Apache YARN:** The resource manager from the Hadoop ecosystem (very common in on-premise setups).
- **Apache Mesos:** A general-purpose cluster manager (less common now).
- **Kubernetes (K8s):** The modern, cloud-native standard for container orchestration (increasingly popular).

2. Driver Node (The Foreman)

This is the process where your `main()` function runs (e.g., the JVM where your PySpark script executes).

Role:

- It holds the `SparkSession` object (the entry point to Spark functionality).
- It converts the user's application code into a directed acyclic graph (DAG) of tasks.
- It schedules and distributes these tasks to the Executors.
- It tracks the progress of tasks and collects the final results.

3. Executor Nodes (The Workers)

These are processes that run on the worker nodes in the cluster and execute the tasks assigned by the Driver.

Role:

- **Execute Tasks:** They run the individual computation tasks (e.g., map, filter, reduce) on data partitions.
- **Store Data:** They hold data in memory or disk and cache it for faster access (e.g., when using `cache()` or `persist()`).
- **Report Back:** They send status and results back to the Driver.

Worker Node:

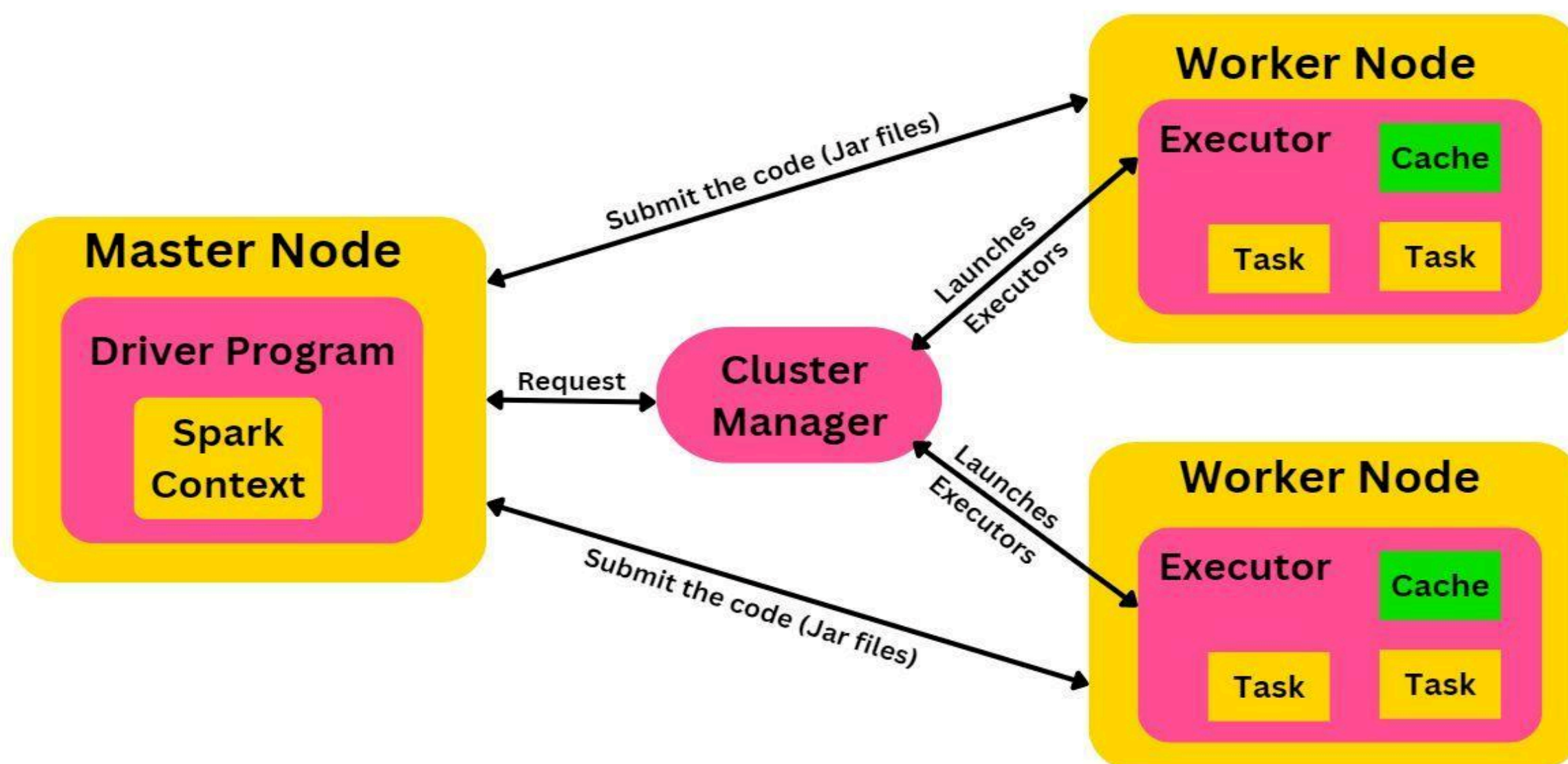
This is a physical or virtual machine in the cluster. Each Worker Node can host one or more **Executor** processes.

How a Spark Cluster Works: Step-by-Step

Let's follow the lifecycle of a PySpark job:

1. **Submission:** You submit your PySpark application (e.g., `spark-submit my_script.py`) to the **Cluster Manager**.
2. **Driver Startup:** The Cluster Manager starts the **Driver Program** on a node in the cluster.
3. **Resource Negotiation:** The Driver talks to the Cluster Manager and requests resources (CPU cores, memory) for its **Executors**.
4. **Executor Launch:** The Cluster Manager launches **Executor** processes on the Worker Nodes.
5. **Task Execution:**
 - The Driver breaks down your code into a logical plan and then an optimized physical plan (a DAG of stages and tasks).
 - The Driver sends the **Tasks** to the Executors.
 - Each Executor runs its tasks on a **partition** (a small chunk) of the overall data, in parallel.
6. **Result Collection:** The Executors return their results to the Driver.
7. **Shutdown:** Once the job is complete, the Cluster Manager shuts down the Executors. The Driver may also shut down, depending on the deployment mode.

Visual Overview of a Spark Cluster



Deployment Modes

Defines *where* the Driver runs.

- **Client Mode:**

The Driver runs on the machine where you ran spark-submit (e.g., your laptop).
If you close your laptop, the job fails. Good for interactive work like notebooks.

- **Cluster Mode:**

The Driver runs *inside* the cluster on a worker node.

This is more robust for production jobs, as the client machine can disconnect without killing the job.

Data Partitioning

- Spark breaks large datasets into smaller chunks called **partitions**.
- Each partition is processed by a single task on a single executor.
- **More partitions = more parallelism**, but also more overhead.
Proper partitioning is key to performance.

Fault Tolerance

- If an Executor fails (e.g., its node crashes), the Cluster Manager will restart it on another node.
- The Driver will re-schedule the tasks that were lost on the new Executor.
- If the Driver fails, the entire application fails (though Spark has tools for checkpointing to recover some state).

Summary

- A **Spark Cluster** is a set of computers (nodes) working together to run Spark applications in parallel.
- It uses a **master-slave architecture** with a **Cluster Manager** (master), a **Driver** (foreman), and many **Executors** (workers).
- The cluster’s power comes from distributing both **data** (via partitioning) and **computation** (via tasks) across all the Executors.
- Managed platforms like **Databricks** and **Azure Synapse** completely abstract away the complexity of setting up and managing this cluster — you just define how much computing power you want, and they provide it.

Basic PySpark Terminologies

Term	Meaning	One-Line Definition / Use
Spark	Distributed processing engine	Used to process big data across multiple machines.
PySpark	Python API for Apache Spark	Allows writing Spark code in Python.
Cluster	Group of machines (nodes)	Executes Spark jobs in parallel.
Driver Program	Main controller of execution	Converts user code into tasks and distributes them.
Executor	Worker process	Executes tasks on worker nodes and stores results.
Application	Spark job submitted to cluster	Whole program or set of operations.
Job	Represents the entire action request	Triggered when an <i>action</i> is called.
Stage	A step in job execution	Created based on shuffle boundaries.
Task	Smallest execution unit	Runs on executors in parallel.
Session (SparkSession)	Entry point for PySpark	Used to interact with Spark and create DataFrames.
RDD (Resilient Distributed Dataset)	Low-level API for distributed data	Immutable, fault-tolerant distributed data.
DataFrame	Distributed table-like data	Most used structured data representation in PySpark.
Schema	Structure of DataFrame	Defines column names and data types.
Dataset	Typed distributed data object	Available in Scala/Java; Python uses DataFrame.
Transformation	Operation creating new DataFrame	Lazy execution (e.g., <code>select</code> , <code>filter</code> , <code>map</code>).

Action	Executes transformations & returns result	Triggers a job (e.g., show, count, collect).
Lazy Evaluation	Spark waits until an action is called	Optimizes execution plan before running.
Catalyst Optimizer	Query optimizer	Generates efficient execution plan.
Tungsten Engine	Physical execution engine	Optimizes memory + CPU usage.
Partition	Logical chunk of data	Spark processes partitions in parallel.
Shuffle	Data movement between partitions	Expensive step involving data redistribution.
Broadcast	Send small dataset to all executors	Used to optimize joins (broadcast join).
Checkpointing	Save data to reliable storage	Used to break lineage and avoid long DAGs.
Persist / Cache	Store data in memory	Used when DataFrame is reused multiple times.
DAG (Directed Acyclic Graph)	Flow of operations	Represents transformations path before execution.

Key points

- Transformations are lazy, actions are not.
- DataFrame is the most commonly used API in PySpark.
- Spark Session is always required to start PySpark.

What is RDD in PySpark

RDD (Resilient Distributed Dataset) is the *core* data structure of Apache Spark.

RDD = Immutable + Distributed + Fault-tolerant collection of data across cluster

Breakdown of the term:

Word	Meaning
Resilient	Fault tolerant — if a node fails, data is automatically recomputed from lineage.
Distributed	Data is split across multiple machines (nodes/partitions).
Dataset	Collection of records (like a list or array) but distributed.

Key characteristics of RDD

Feature	Explanation
Immutable	Once created, cannot be changed. New RDD is created on every transformation.
Lazy evaluation	Transformations are not executed until an action is called.
Fault-tolerant	If data is lost, Spark can recompute using <i>lineage</i> (history of transformations).
Distributed	Data is split into partitions and processed in parallel.

Low-level API	User controls partitioning and operations like map, filter, reduce.
----------------------	---

What is Shuffle in PySpark

Shuffle is the process of **re-distributing data across partitions** (and nodes) in the cluster, usually because of transformations that require data to be grouped or combined based on a key.

In simple words: Shuffle = data movement across nodes.

What is a Transformation

A **Transformation** is an operation on a DataFrame/RDD that produces **another DataFrame/RDD**.

Transformations are **lazy**, meaning Spark does not run them until an **action** is called (show(), count(), etc.)

Types of Transformations

There are **two main types**:

1. Narrow Transformations

Data required to compute the result comes from **a single partition**.

No data movement (no shuffle).

2. Wide Transformations

→ Data needs to be **re-distributed across partitions** (shuffle happens).

→ Spark must move data over the network.

What is an Action in PySpark

An **Action** is an operation that **triggers execution of the entire Spark job**.

→ When you call an action, Spark executes all the previous **lazy transformations** and returns a result to the driver or writes the output to storage.

Transformations build the plan; Actions execute the plan.

Task01- create two Databricks account

- One free tier (also called community edition)

Compute is serverless on free tier

- Another on cloud premium
Compute is dedicated

Control Plane, Compute Plane, Workspace, and Cloud Storage in Databricks:

1. Control Plane

Managed by Databricks (not customer)

It is the **brain** that manages the Databricks environment.

Responsible for:

- Cluster management (start, stop, scale)
- Job scheduling
- Notebook management
- User authentication (login, permissions)
- Monitoring, UI, logs, metadata, query history

It **does NOT store customer data**

It only manages and controls operations

2. Compute Plane (Data Plane)

Runs in the customer's cloud account (AWS / Azure / GCP)

This is where **actual code execution happens**

Components:

- Spark **Driver Node** (coordinates execution)
- Spark **Worker Nodes** (process data in parallel)
- Runs your notebooks, jobs, Spark tasks, ML training, etc.

This is where data is **processed**

Connected to your cloud storage to read/write data

3. Workspace

It is the **user interface / development environment** you work in.

Contains:

- Notebooks (Python, SQL, Scala, R)

- Jobs, pipelines
- Dashboards
- Repos (Git integration)
- Libraries

Used for **building and managing work**
Not used for storing raw data

4. Cloud Storage (Your Data Lake)

Databricks does **NOT** store data permanently.

Data is stored in your cloud account:

Cloud	Storage
AWS	S3
Azure	ADLS Gen2 / Blob Storage
GCP	GCS

Also supports:

- **Delta Lake** (ACID, schema enforcement, time travel)
- DBFS (Databricks File System) – abstraction over cloud storage

Stores raw & processed data
Highly scalable & secure

How They Work Together

Workspace → Control Plane → Compute Plane → Cloud Storage
(User writes code) (Manages) (Executes code) (Reads/Writes data)

Quick Summary Table

Component	Managed By	Purpose	Stores Data?
Control Plane	Databricks	Cluster, jobs, UI, auth	✗ No
Compute Plane	Customer Cloud	Executes Spark code	✗ No (temporary cache only)
Workspace	Databricks	Development (notebooks, jobs)	✗ No
Cloud Storage	Customer Cloud	Permanent data storage	✓ Yes

Databricks Magic Commands

%md in Databricks

%md is a **magic command** used to write **Markdown** text in a Databricks notebook.

Purpose:

- Add **notes, headings, documentation**
- Create **lists, tables, links, images**
- Make notebooks **structured and readable**

Example:

```
%md
```

```
# Databricks Tutorial
```

```
## Introduction
```

- Databricks is built on ****Apache Spark****
- Used for ****Big Data, AI, and Analytics****

```
### Features
```

1. Fast processing
2. Collaboration
3. Delta Lake support

```
[Go to Google](https://www.google.com)
```

Output will show formatted text like:

Databricks Tutorial

Introduction

- Databricks is built on **Apache Spark**
- Used for **Big Data, AI, and Analytics**

Features

1. Fast processing
2. Collaboration
3. Delta Lake support

[Go to Google](https://www.google.com)

Common Markdown Use

Markdown	Result
# Heading	Big Heading
## Sub Heading	Medium Heading
bold	bold
italic	<i>italic</i>
- bullet	• List
1. item	Numbered list
[text](link)	Link

Tip

Use %md to document your code and explain steps in the notebook.

language

Command	Purpose
%python	Run Python code
%sql	Run SQL queries
%scala	Run Scala code
%r	Run R code

Category	Example
Language	%python, %sql
File System	%fs ls, %fs rm, %fs list
Run Notebook	%run
Package Install	%pip (to install python library)
Widgets	dbutils.widgets
shell commands	%sh

Examples

%fs

ls / # list file in home dir

or

%fs ls / #can be in one line too

Example

```
%sh ps
```

Example

```
%pip list
```

Examples

%run in Databricks

%run is a **magic command** used to execute another notebook from the current notebook.

Syntax

```
%run /Path/To/Notebook
```

Task02

Create a sample notebook

Syntax-

```
env = 'test'
```

create a function as to find python version and databricks function

syntax-

```
import sys
import os

def get_environment_info():
    python_version = sys.version.split("\n")[0]
    # Databricks runtime version from environment variable
    runtime = os.environ.get(
        "DATABRICKS_RUNTIME_VERSION",
        "Not found"
    )
    return {
        "python_version": python_version,
        "databricks_runtime": runtime
    }

info = get_environment_info()
```

```
display(info)
```

\$ call sample notebook from current Notebook

Syntax-

```
%run "fullpathofsamplenotebook"
```

Test

env

call function with function name

```
get_environment_info()
```

Databricks Utilities (dbutils)

dbutils is a set of built-in utility commands in Databricks notebooks that help you interact with:

- Files (DBFS / cloud storage)
- Notebooks (run, exit, parameters)
- Secrets
- Widgets (input parameters)

Example

```
%fs ls / # good for quick task

dbutils.fs.ls('/') # good for programmatic task

display(dbutils.fs.ls('/'))
```

Main Categories of Databricks Utilities

Utility	Purpose
dbutils.fs	File operations (List, Copy, Remove, Mount, etc.)
dbutils.notebook	Notebook execution (run, exit)

dbutils.widgets	Create input parameters (UI widgets)
dbutils.secrets	Access secret values securely
dbutils.library	Install Python/JAR libraries (limited use)

1. File System Utilities (dbutils.fs)

```
dbutils.fs.ls("/FileStore/")
dbutils.fs.mkdirs("/FileStore/myfolder")
dbutils.fs.rm("/FileStore/myfolder", True) # True = recursive delete
dbutils.fs.cp("/FileStore/source.txt", "/FileStore/dest.txt")
dbutils.fs.head("/FileStore/data.csv")
```

2. Notebook Utilities (dbutils.notebook)

Run another notebook:

```
dbutils.notebook.run("/Users/pawan/helpers", 60)
```

Exit notebook:

```
dbutils.notebook.exit("Notebook Completed Successfully")
```

3. Widgets (Parameters in Notebook UI)

Databricks Widgets –

Widgets are input parameters in Databricks Notebooks used to make notebooks **dynamic**, reusable, and interactive.

Create a text input widget:

```
dbutils.widgets.text("username", "", "Enter name")
```

Get widget value:

```
name = dbutils.widgets.get("username")
print(name)
```

Remove all widgets:

```
dbutils.widgets.removeAll()
```

4. Secrets Utilities

List secret scopes:

```
dbutils.secrets.listScopes()
```

Get secret value:

```
dbutils.secrets.get(scope="myscope", key="dbpassword")
```

5. Library Utilities

Install a Python library:

```
dbutils.library.installPyPI("pandas")
dbutils.library.restartPython()
```

Quick Summary

Command	Example
File list	<code>dbutils.fs.ls("/FileStore/")</code>
Run notebook	<code>dbutils.notebook.run("/Path", 90)</code>
Create widget	<code>dbutils.widgets.text("x","", "Enter x")</code>
Get secret	<code>dbutils.secrets.get("scope","key")</code>
Make folder	<code>dbutils.fs.mkdirs("/newfolder")</code>

Important Notes

- Works only in Databricks Notebook
- Not available in local Python
- Helps automate workflows

Difference between dbutils.fs and dbutils utilities

Short Answer

- **dbutils** is the **main utility package** in Databricks.
- **dbutils.fs** is **one module inside dbutils**, used only for **file system operations**.

Detailed Comparison

Feature	dbutils (Utilities)	dbutils.fs (File System Utility)
What is it?	A collection of utility modules	A sub-module of dbutils
Purpose	Notebook automation, secrets, parameters, workflows, libraries, file ops	Only file handling in DBFS/cloud storage

Scope	Wide (widgets, notebooks, secrets, libraries, file system)	Limited to file operations
Examples include	widgets, notebook, secrets, library, fs	ls, cp, rm, mkdirs, head, mount...
Can run notebooks?	✓ Yes (dbutils.notebook.run)	✗ No
Can read secrets?	✓ Yes (dbutils.secrets.get)	✗ No
Can create widgets?	✓ Yes (dbutils.widgets.text)	✗ No
Can list DBFS files?	✓ Yes but via dbutils.fs.ls	✓ Yes
Category	Parent utility package	Child module focused on storage

What is Azure Repos + Databricks Integration

It allows you to:

- Version control Databricks notebooks
- Collaborate using **Git**
- Maintain CI/CD pipelines
- Keep code safe in a central repository

Azure Repos supports both **Git** and **TFVC**, but Databricks uses **Git only**.

Steps to Integrate Azure Repos with Databricks

1. Get Azure DevOps Repo URL

1. Go to: <https://dev.azure.com/>
2. Select your **Organization** → **Project** → **Repos**
3. Copy the **Clone URL (HTTPS)**
- 4.

Example:

https://dev.azure.com/ORG_NAME/PROJECT_NAME/_git/REPO_NAME

2. Create Personal Access Token (PAT)

1. In Azure DevOps → Click profile icon (top right)
2. Go to **Personal Access Tokens**
3. Click **New Token**
4. Enable scope:
 - o ☒ Code → **Full**
5. Copy the token (you will not see it again!)

3. Connect in Databricks

1. Go to Databricks workspace → **User Settings**
2. Open **Git Integration**
3. Enter:
 - o **Git Provider** → Azure DevOps Services
 - o **Username** → your email/username
 - o **Password/Token** → Paste PAT token
4. Click **Save**

4. Link a Databricks Project to the Repo

1. Open a folder/workspace in Databricks
2. Click **Git** icon (top right)
3. Select **Git: Clone**
4. Paste repo URL
5. Choose branch (main/dev)
6. Click **Clone**

Now your Databricks workspace is connected to Azure Repos!

Git Operations Supported in Databricks

Operation	Supported
Clone Repo	☒
Create Branch	☒
Commit	☒
Push	☒
Pull	☒
Merge	☒ (do in Azure DevOps)

Pull Requests

✗ (do in Azure DevOps)

Note: Merge and PR must be done inside Azure Repos, not Databricks.

Best Practices

- Use **Feature branches** (not direct main commits)
- Raise **Pull Requests** for review
- Use **Azure Pipelines** for deployment
- Store configs in **.json/.yaml files**, not hard-coded in notebooks
- Use **Databricks Secrets** for credentials

What is Unity Catalog? (Databricks)

Unity Catalog is **Databricks’ centralized governance & metadata management layer** for data and AI. It provides **security, data sharing, auditing, and lineage** across Databricks workspaces.

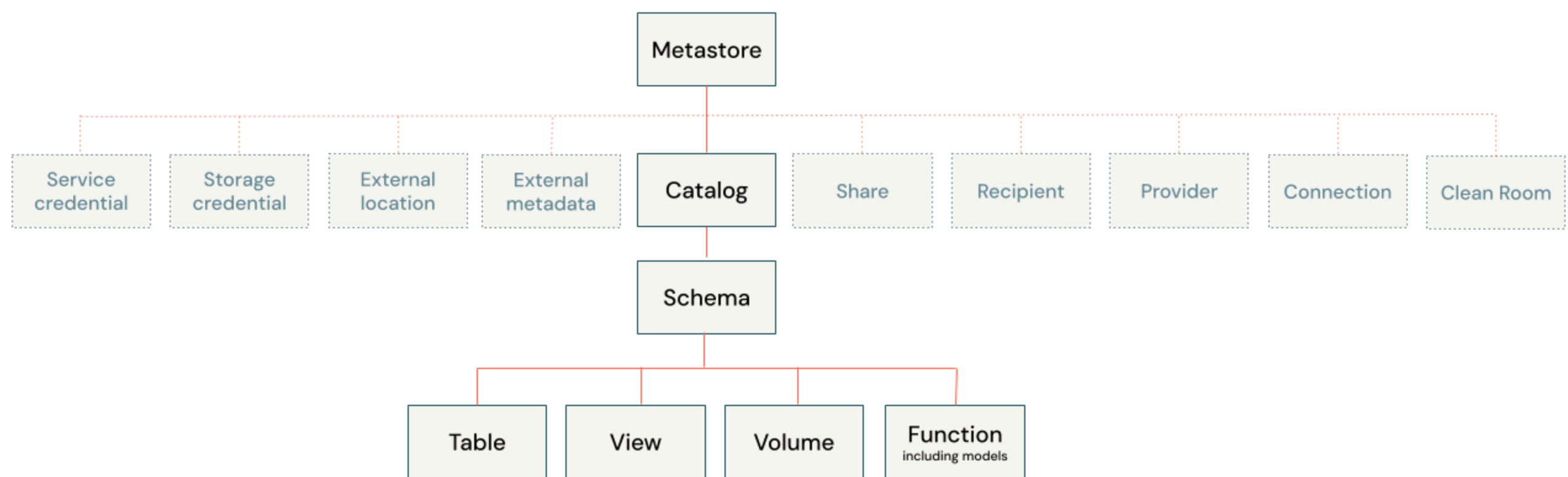
In Simple Words:

Unity Catalog is the **security + organization + tracking system** for all your data, tables, models, and files in Databricks.

Key Benefits

Feature	Benefit
Centralized Access Control	One place to manage permissions
Data Lineage	See where data came from and how it is used
Auditing	Track who accessed/modified data
Data Sharing (Delta Sharing)	Share data securely outside your org
Catalog → Schema → Table structure	Organized data like a database
Row & Column-Level Security	Restrict sensitive data
Unity across workspaces	Same governance for all workspaces

How Data is Organized in Unity Catalog



Catalog → Schema (Database) → Table / View / Volume

Example:

```

sales_catalog
├── customer_schema
│   └── customer_table
  
```

Security Model

Level	Can control access?
Catalog	✓
Schema	✓
Table	✓
Column	✓ (column masking allowed)
Row	✓ (row-level filters)

Auditing & Lineage

You can track:

- Who **read or modified** data?
- Which job or notebook used which table?
- Full transformation path (source → target)

Works With

- ✓ Delta Lake tables
- ✓ ML models
- ✓ Files (through volumes)
- ✓ Notebooks, Jobs, SQL dashboards
- ✓ External data sources (Snowflake, ADLS, etc.)

Data Sharing

Unity Catalog enables **Delta Sharing**, which allows:

- Secure data sharing **without data copy**
- Internal & external sharing (partners, teams, clients)

Architecture

User → Databricks Workspace → Unity Catalog Governance Layer → Storage (ADLS / S3 / GCS)

Best Use Cases

Use Case	Why Unity Catalog
Large Data Teams	Centralized governance
Sensitive Data	Masking & fine-grained access
Compliance	Audits + lineage
Multiple Workspaces	Unified control
Data Sharing	Delta Sharing

Summary

Unity Catalog is for...	Not for...
Security & governance	Data processing engine
Metadata & access control	Replacing Spark/SQL
Sharing & auditing	Direct storage layer

To check wheather workspace is connected to unity catalog metastore

```
%sql
select current_metastore();
```

it will show the metatore, if it is not attached then it will show the error

task01- access workspace as databricksuser

will get information of metastore attached

Unity catalog overview

Task02- Configure access to storage account

Step-01

01 Create access connector

What is Access Connector for Databricks?

Access Connector for Azure Databricks is a managed identity wrapper provided by Azure that lets Databricks securely connect to Azure Data Lake Storage (ADLS Gen2) or other Azure services without using secrets, keys, or service principals.

In Simple Words

It allows Databricks to access ADLS without storing credentials, using Managed Identity authentication instead.

Why is it used

Without Access Connector	With Access Connector
Need to store secrets (client ID, secret etc.)	No secrets needed
Need Service Principal management	Managed by Azure automatically
Less secure	More secure (no credentials stored)
Extra configuration	Simple & recommended for Unity Catalog

Main Use Case

Used when configuring Unity Catalog storage access, especially to:

- Access ADLS Gen2
- Create External Locations
- Create Managed & External Tables
- Provide secure storage access in Databricks using Managed Identity

Task02 create Azure datalake storage account (premium)

Task03 assign access role (storage blob data contributor) on ADLS assign managed identity to access connector

Task04 create credentials

Databricks portal → catalog → external data → credentials

Task 05 create external location

Create container → create notebook →

Code

```
CREATE EXTERNAL LOCATION [IF NOT EXISTS] location_name
  URL url_str
  WITH (STORAGE CREDENTIAL credential_name)
  [COMMENT comment]
```

```
%sql
CREATE EXTERNAL LOCATION IF NOT EXISTS
`adlsstoraacccdatabricks_test-container`
  URL
'abfss://test-container@adlsstoraacccdatabricks.dfs.core.windows.net/'
  WITH (STORAGE CREDENTIAL accessconnectorfortest)

defaultformat

abfss://<container>@<storage-account>.dfs.core.windows.net/<optional-path>
```

```
%fs ls 'abfss://test-container@adlsstoraacccdatabricks.dfs.core.windows.net/'
```

Databricks Catalog-level commands — these are specific to **Unity Catalog** management

Unity Catalog – Catalog Commands

A **catalog** in Unity Catalog is the highest-level container for your data objects (schemas, tables, views, etc.).

Hierarchy:

Catalog → Schema → Table/View/Function

1. Create Catalog

```
CREATE CATALOG IF NOT EXISTS my_catalog  
COMMENT 'This catalog contains sales and analytics data';
```

- IF NOT EXISTS → prevents errors if catalog already exists
- COMMENT → optional description

2. Show Catalogs

```
SHOW CATALOGS;
```

Shows a list of all catalogs available in Unity Catalog for your workspace.

3. Use Catalog

```
USE CATALOG my_catalog;
```

Sets the active catalog context for your SQL session or notebook.
After this, you can create schemas, tables, etc. within this catalog.

4. Drop Catalog

```
DROP CATALOG IF EXISTS my_catalog CASCADE;
```

- Removes the catalog and all contained schemas/tables
- **CASCADE** ensures everything inside the catalog is deleted as well

5. Describe Catalog

```
DESCRIBE CATALOG my_catalog;  
DESCRIBE EXTENDED CATALOG my_catalog;
```

Shows metadata like owner, comment, creation time, etc.

6. Permissions on Catalog

Grant Permissions

```
GRANT USAGE ON CATALOG my_catalog TO `data_engineers`;  
GRANT CREATE ON CATALOG my_catalog TO `admin_group`;
```

Revoke Permissions

```
REVOKE USAGE ON CATALOG my_catalog FROM `data_engineers`;
```

Show Grants

```
SHOW GRANTS ON CATALOG my_catalog;
```

7. Catalog Ownership

```
ALTER CATALOG my_catalog OWNER TO `new_owner`;
```

Transfers ownership of the catalog.

8. Alter Catalog

```
ALTER CATALOG my_catalog SET COMMENT 'Updated comment for catalog';
```

9. System & Default Catalogs

Catalog Name	Description
main	Default Unity Catalog catalog created automatically
hive_metastore	Legacy default catalog (non-Unity Catalog)
system	Stores system-level tables and audit logs

You can switch catalogs using:

```
USE CATALOG hive_metastore;  
USE CATALOG main;  
USE CATALOG my_catalog;
```

Quick Summary Table

Action	SQL Command
Create Catalog	CREATE CATALOG my_catalog
List Catalogs	SHOW CATALOGS
Use Catalog	USE CATALOG my_catalog
Drop Catalog	DROP CATALOG my_catalog CASCADE
Describe Catalog	DESCRIBE CATALOG my_catalog
Alter Catalog	ALTER CATALOG my_catalog SET COMMENT 'text'
Grant	GRANT USAGE ON CATALOG ...
Revoke	REVOKE USAGE ON CATALOG ...
Change Owner	ALTER CATALOG my_catalog OWNER TO user

Task01 check list of catalogs available

SHOW CATALOGS

Task02 create catalogs

```
%sql
CREATE CATALOG  IF NOT EXISTS  test_catalog
    MANAGED LOCATION
'abfss://psacontainer@psastorageaccoun01t.dfs.core.windows.net/'
```

Task 03 SHOW CATALOGS

Task04 show curren catalogs in use

```
select current_catalog();
```

task 05 switch to other catalog

```
%sql
use catalog test_catalog
```

task06 create schema

```
%sql
CREATE SCHEMA IF NOT EXISTS schema_name
    MANAGED LOCATION
    'abfss://psacontainer@psastorageaccoun01t.dfs.core.windows.net/'
```

Task 07 Show shema

```
%sql
show schema
```

task 08 create multiple schema

```
%sql
CREATE SCHEMA IF NOT EXISTS schema_default
    MANAGED LOCATION
    'abfss://psacontainer@psastorageaccoun01t.dfs.core.windows.net/';

CREATE SCHEMA IF NOT EXISTS schema_01
    MANAGED LOCATION
    'abfss://psacontainer@psastorageaccoun01t.dfs.core.windows.net/';

CREATE SCHEMA IF NOT EXISTS schema_02
    MANAGED LOCATION
    'abfss://psacontainer@psastorageaccoun01t.dfs.core.windows.net/';

CREATE SCHEMA IF NOT EXISTS schema_03
    MANAGED LOCATION
    'abfss://psacontainer@psastorageaccoun01t.dfs.core.windows.net/';
```

Task09 show schemas

```
%sql
show schemas
```

task 10 create volume

```
USE CATALOG test_catalog;
```

```
USE SCHEMA schema_default;

CREATE EXTERNAL VOLUME IF NOT EXISTS volume_name
  LOCATION
  'abfss://psacontainer@psastorageaccount01t.dfs.core.windows.net/volumes/volume_name/';
```

Databricks Workflows (Jobs)

Databricks **Workflows (Jobs)** are a fully managed way to **orchestrate, schedule, and run** data tasks such as notebooks, Python scripts, SQL queries, pipelines, and ML workflows — all in a reliable, production-ready manner.

What is a Databricks Job / Workflow

A **Job** (also called **Workflow**) in Databricks is used to:

- ✓ Automate data pipelines
- ✓ Schedule notebooks or scripts
- ✓ Run ETL / ELT tasks
- ✓ Trigger Delta Live Tables pipelines
- ✓ Run ML training/inference tasks
- ✓ Connect multiple tasks with dependencies

It turns your development notebook or script into a **repeatable, production-ready** workflow.

Key Components of Databricks Jobs

1. Tasks

Each job contains one or more **tasks** like:

- Notebook task
- Python script task
- SQL query task
- dbt task
- Delta Live Tables pipeline task
- JAR task

2. Cluster Options

You can choose:

- **Job Cluster** (preferred — auto spin up & shut down) **note- expensive**
- **Existing Interactive Cluster**

3. Scheduling

You can set:

- Cron schedule
- Every X minutes/hours/day
- Event-based triggers (including **file arrival trigger** for ADLS / S3)

4. Dependencies

You can define:

- Task A → Task B
- Task C waits for A & B
- Parallel execution

This makes it a full workflow orchestrator.

Why Use Databricks Workflows

- ✓ No need for external orchestrators like Airflow
- ✓ Built-in retry, email alerts, logging
- ✓ Can handle complex DAG pipelines
- ✓ Unified UI for monitoring & logs
- ✓ Low-cost Job Clusters (auto-terminate)

Simple Example: Single Task Job

Task: Run a notebook

Schedule: Daily 2 AM

Cluster: Job cluster with 1 driver + 2 workers

Outcome: Automatically refresh your Delta tables daily.

Triggers in Databricks

Triggers in Databricks

In Databricks, **triggers** define *when* and *how often* a pipeline, job, or table processing task should run.

There are mainly **two places** where triggers are used:

1. Triggers in Delta Live Tables (DLT)

DLT supports two trigger modes:

a) Continuous Trigger

- Runs the pipeline **24/7 continuously**.
- Best for **streaming data** or near-real-time processing.
- Automatically processes new data as soon as it arrives.

Syntax:

```
LIVE.trigger(continuously=True)
```

b) Scheduled Trigger

- Runs **on a fixed schedule**, like hourly/daily.
- Best for **batch workloads**.

Syntax:

```
LIVE.trigger(interval="1 hour")
```

Examples:

✓ Every 1 hour

```
LIVE.trigger(interval="1 hour")
```

✓ Every 10 minutes

```
LIVE.trigger(interval="10 minutes")
```

✓ Run daily

```
LIVE.trigger(interval="1 day")
```

2. Triggers in Databricks Workflows (Jobs)

In **Workflows**, triggers decide when the job runs.

Types of triggers:

a) Schedule Trigger

- Cron or hourly/daily schedule.
- Example: run everyday at 7 AM.

b) Continuous Trigger

- Job runs once the previous run finishes.
- Used for continuous ETL.

c) File Arrival Trigger (Auto Loader)

- Automatically triggers when a new file arrives in a storage location.
- Perfect for streaming ingestion.
-

Example using Auto Loader:

```
df = spark.readStream.format("cloudFiles") \
    .option("cloudFiles.format", "json") \
    .load("path")
```

This automatically triggers pipeline on file arrival.

Summary Table

Trigger Type	Used In	Purpose
Continuous Trigger	DLT	Real-time continuous processing
Interval Trigger	DLT	Scheduled batch processing
Schedule Trigger	Workflows	Run jobs based on cron/time schedule
File Trigger	Workflows / Auto Loader	Run job when files arrive
Continuous Job Trigger	Workflows	Re-run automatically when previous ends